

Markov chain Monte Carlo, part 1

Covered by chapter 11 in the textbook and these notes.

In many applications (e.g. physics, statistics/machine learning) one is confronted by the following problem: It is known that some random vector, say $\boldsymbol{\theta}$, has probability density which is *proportional* to some function $g(\boldsymbol{\theta})$, i.e. there exist some constant C so that

$$\pi(\boldsymbol{\theta}) = \frac{1}{C}g(\boldsymbol{\theta}), \quad \text{where } C = \int g(\boldsymbol{\theta})d\boldsymbol{\theta}.$$

Further, you are tasked with finding e.g. $E(\boldsymbol{\theta})$, $E(r(\boldsymbol{\theta}))$, some quantile of $\boldsymbol{\theta}$ and so on.

Examples:

- Bayesian statistics: θ has the "posterior distribution" of the parameters, reflecting all there is to be known about the parameter after data has been observed. $g(\theta) = L(\theta)p(\theta)$ is the product of likelihood and prior. C = "evidence".
- Physics/Chemistry: θ has Boltzmann distribution of system, $\log g$ is the negative Hamiltonian.
- Machine learning: Boltzmann machines, Bayesian neural nets, Gaussian processes...
- And so on...

MCMC

When $\boldsymbol{\theta}$ has dimension more than say 2 or 3, finding C or evaluating integrals, e.g. $\int r(\boldsymbol{\theta})g(\boldsymbol{\theta})d\boldsymbol{\theta}$ using conventional numerical integration methods is close to hopeless.

Since we are accustomed to Monte Carlo methods by now, it makes sense to look for **ways of generating random variables with density $\propto g(\boldsymbol{\theta})$** , and use these to calculate the sought expectations, quantiles etc.

Markov chain Monte Carlo (MCMC) methods are the answer to this question; they are algorithms for generating discrete time Markov processes which have $\pi(\boldsymbol{\theta})$ as the stationary distribution, while *requiring only the ability to evaluate g (and not C)*.

”The price to pay for not being able to evaluate C is that random draws will in general no longer be independent”.

Random walk Metropolis Hastings (RWMH): algorithm

The most basic, and probably most frequently used MCMC method is RWMH: Suppose some covariance matrix Σ and an initial position $\boldsymbol{\theta}_0$ has been chosen in advance. Then the algorithm proceeds by the following steps:

1. set $t \leftarrow 1$
2. simulate $\boldsymbol{\theta}^* \sim N(\boldsymbol{\theta}_{t-1}, \Sigma)$.
3. simulate $u \sim \text{uniform}(0,1)$.
4. if $u < \min(1, \pi(\boldsymbol{\theta}^*)/\pi(\boldsymbol{\theta}_{t-1}))$ set $\boldsymbol{\theta}_t = \boldsymbol{\theta}^*$.
Otherwise set $\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1}$.
5. set $t \leftarrow t + 1$ and go back to step 2.

Under very general conditions, this algorithm will generate a Markov process $\{\boldsymbol{\theta}_t\}_{t=1}^T$ which has $\pi(\boldsymbol{\theta})$ as the stationary distribution.

See example `mcmc_1.R`

Random walk Metropolis Hastings (RWMH): density kernels

Recall the motivation for discussing MCMC in the first place: Wish to draw samples with density $\propto g(\boldsymbol{\theta})$, where the normalization constant C is not available.

Notice that the RWMH algorithm only evaluates the target density π in the fraction in the accept probability:

$$\alpha = \min \left(1, \frac{\pi(\boldsymbol{\theta}^*)}{\pi(\boldsymbol{\theta}_{t-1})} \right)$$

Rewrite:

$$\alpha = \min \left(1, \frac{g(\boldsymbol{\theta}^*)/C}{g(\boldsymbol{\theta}_{t-1})/C} \right) = \min \left(1, \frac{g(\boldsymbol{\theta}^*)}{g(\boldsymbol{\theta}_{t-1})} \right)$$

I.e. Normalization constant not needed, it suffices to know the target density only up a constant of proportionality!!!

See example `mcmc_1.R`

Example from Bayes notes

Recall

$$\begin{aligned} g(\boldsymbol{\theta}) &= L(\boldsymbol{\theta})p(\boldsymbol{\theta}) \\ &= \frac{\exp\left(-\frac{(\mathbf{y}-\boldsymbol{\theta})^2}{2\sigma^2}\right)}{\sqrt{2\pi\sigma^2}} \frac{\exp\left(-\frac{\boldsymbol{\theta}^2}{200}\right)}{\sqrt{200\pi}} \end{aligned}$$

Through non-trivial computations, we found C and $\pi(\boldsymbol{\theta}) = p(\boldsymbol{\theta}|\mathbf{y})$, but simulation from $p(\boldsymbol{\theta}|\mathbf{y})$ could also be done numerically using MCMC.

See example `mcmc_1.R`

Use the MCMC output to approximate probability of slippery hill and temperature range for jacket choice.

See example `mcmc_1.R`

Tuning the RWMH

The choice of the covariance matrix Σ used in the proposal of the RWMH determines the efficiency of the algorithm.

Accept probability = $E(\alpha)$

Rule of thumb 1 : for moderate dimension, choose Σ so that accept rate is around 20% - 40%.

Rule of thumb 2 : for dimension > 1 , Σ should be close to proportional to the covariance matrix of θ .

Too "small" Σ : Process moves very slowly around the support of π , results in high accept probability.

Too "large" Σ : Almost all proposals are inconsistent with the target distribution, results in low accept probability.

See example `mcmc_1.R`

Using MCMC output

Unlike earlier applications of Monte Carlo methods, where we are mainly interested in computing integrals, with MCMC methods, the output is typically **a representation of some probability distribution** (the posterior in Bayesian applications).

This representation may be used in different ways:

- Computing expectations and quantiles, etc of the target distribution (using e.g. $E(h(\boldsymbol{\theta})) \approx \frac{1}{T} \sum_{t=1}^T h(\boldsymbol{\theta}_t)$)
- Finding representation of the distribution of some transformed quantity, say $\eta = k(\boldsymbol{\theta})$ (by applying k to all samples, i.e. $\eta_t = k(\boldsymbol{\theta}_t)$, $t = 1, \dots, T$)
- Finding the distribution of future observations, conditional on priors and the observations we have so far.
- Much more...